A Novel Hierarchical Parallelism for Accelerating NIDS Using GPUs

Cheng-Hung Lin and Cheng-Hung Hsieh
Department of Electrical Engineering, National Taiwan Normal University
Taipei, Taiwan, R.O.C
Email: brucelin@ntnu.edu.tw, Tel: +886-2-7734-3432
Email: chadcoco1444@gmail.com, Tel: +886-989-365-782

Abstract

Multi-string matching has been widely used in network intrusion detection systems to detect network attacks and malicious network packets by matching packet contents with thousands of attack patterns. Due to the rapid increase of network attacks and network speeds, multi-string matching faces higher performance and throughput challenges. In order to improve the performance and throughput of multi-string matching, this paper presents a hierarchical parallelism that accelerates multi-string matching on multiple GPUs. The hierarchical parallelism consists of three layers of parallelism. From top to bottom, the first layer is the data parallelism on multiple GPUs, the second layer is the task parallelism on a single GPU, and the last layer is the data parallelism of the Aho-Corasick algorithm. Experimental results show that the hierarchical parallelism on a machine featured with four Titan X GPUs achieves up to 70 Gbps of throughput, more than 40 times faster than the Aho-Corasick algorithm used in Snort. As the number of GPUs increases, the throughput of the hierarchical parallelism will increase. In addition, the proposed approach adopts perfect hashing to construct state machines and achieves up to 99.2% of memory reduction on Snort. Finally, the proposed hierarchical parallelism is implemented in the open source network intrusion detection system, Snort.

Key words: network intrusion detection systems, graphics processing units, multiple string matching, Aho-Corasick algorithm

I. Introduction

Due to the increasing popularity of the Internet today, network security is becoming more and more important. Network Intrusion Detection Systems (NIDS) are designed to monitor malicious activity or policy violations on networks or systems to detect malicious network attacks. Because of the exponential growth of the number and type of network attacks, the load of the detection system is getting larger as the rule set increases. At the same time, advances in hardware have also increased network bandwidth, and NIDS must improve the overall performance of the system to meet current network requirements.

Snort [2] is a powerful and lightweight open source network intrusion detection system with good scalability and portability. It has the ability to analyze instantaneous network traffic, log packets on the network, perform analysis of network protocols including TCP, UDP, and ICMP, and search for attack patterns. It can detect a variety of attacks, real-time alerts on attacks such as buffer overflows, port scans, CGI attacks, SMB probes, and attempts to probe fingerprints of the operating system. Since its publication in 2002, Snort has used the Aho-Corasick (AC) [1] algorithm to match multiple string patterns.

According to OSEC (Open Security Evaluation Criteria) verification report [3], Snort's performance is about 750 Mbps, of which the execution time of multiple string matching accounted for 41.68%. In other words, Snort faces a big challenge to accommodate the increasing number of attack patterns and meet the network speed requirements. Therefore, many hardware approaches have been proposed, including FPGA [4] [10], ASIC [5], and GPUs [6] to improve the performance of Snort. In our past researches, we have had some preliminary results in accelerating string matching using GPUs. We propose the Parallel Failureless Aho-Corasick (PFAC) [7] [8] algorithm to accelerate the traditional Aho-Corasick algorithm using GPUs. We also propose using a perfect hash to compress the state transition table and achieve significant memory reduction. In this paper, we extend our preliminary study and propose a hierarchical parallelism mechanism to optimize Snort and reduce its memory requirements. The hierarchical parallelism consists of three layers of parallelism. From top to bottom, the first layer is the data parallelism on multiple GPUs, the second layer is the task parallelism on a single GPU, and the third layer is the data parallelism of the Aho-Corasick algorithm. Experimental results show that the hierarchical parallelism on a machine with four Titan X GPUs achieves up to 70 Gbps of throughput, more than 40 times faster than the traditional AC algorithm used in Snort. As the number of GPUs increases, the throughput of hierarchical parallelism will increase. In addition, Snort uses linked lists to construct state machines, leading to inefficient traversal of the AC state machines and memory requirements for storing pointers. We propose to use perfect

hashing [9] to store the state machines and achieves up to 99.2% of memory reduction on Snort. Finally, the proposed hierarchical parallelism is implemented in Snort, an open source network intrusion detection system.

II. Related Work

Common NIDS searches and compares packet contents, including headers and payloads of different protocol packets, and determines whether the packet contains a possible virus signature or attack. Therefore, checking a large number of packets on the network requires a huge amount of computing power. Depicts the flow of handling packets in Snort 2.9.x which is the current most popular open source NIDS. First, Snort retrieves packets from a network or network interface using libpcap or other libraries, and decodes the captured packets into Snort's data structure through the Decode module. The preprocess module then modifies or analyzes the packet. The detection module is to search the packet contents against the compiled attack patterns. If an attack or feature is detected, Snort sends the alert back to the user.

The process described in snort is a single-thread NIDS architecture. In recent years, many architectures have been proposed to accelerate Snort. For example, Para-Snort [5] proposes to assign packets to different hierarchical modules which use multiple threads with shared memory to optimize Snort. Experimental results show that the system achieves throughput of 600 Mbps under the HTTP protocol. Due to the huge parallel computing power of GPUs, Gnort [6] improve the traditional Aho-Corasick algorithm using GPUs and achieve a system throughput of 2.3 Gbps. With the increasing speed of the network, the Snort faces a large challenge in the performance and throughput.

III. Hierarchical parallelism

The original Snort processes one packet at a time, and all packets are processed sequentially. This strategy does not apply to GPUs as it is not cost-effective to process single and small data packets on GPUs. Since GPUs can handle large amounts of data at the same time, we propose that batch technology put the packets in buffers and then process each buffer on a batch basis. In order to achieve batch processing, we propose a hierarchical parallelism architecture which adopts multiple GPUs to optimize Snort with data and task parallelism. Fig. 1 shows the proposed hierarchical parallelism architecture which consists of three levels of parallelism.

A. Data parallelism on multiple GPUs

The first level is the data parallelism on multiple GPUs where each data buffer is allocated to its associated GPU. Different protocol network packets are sent to a specific buffer. In this stage, we record the order of the packets, the location of the packet, to facilitate the subsequent post-processing action. When the buffer is full or the maximum

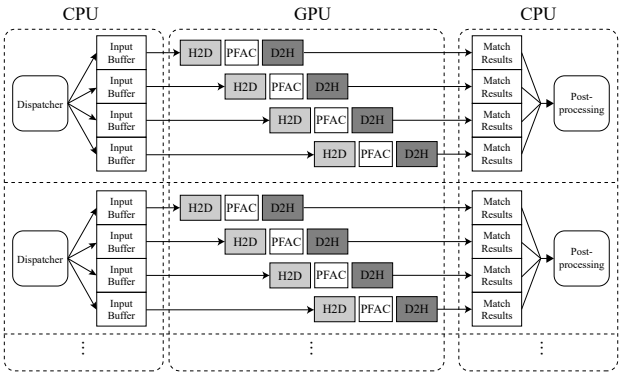

Fig. 1: The hierarchical parallelism mechanism

elapsed time is up, the contents of the buffer will be moved to the GPU. To meet real-time requirements, the buffer size is set to 300 MB and the maximum elapsed time is set to 0.45 seconds.

B. Task parallelism on a single GPU

The second level is the task parallelism which implements pipelining to utilize the resources of GPUs. A typical GPU process consists of three tasks, moving data from CPU to GPU, GPU kernel function, and moving data from GPU to CPU. CUDA provides a streaming technology that splits the input into multiple streams and sends them one by one to the GPU, which is so called pipelining.

C. Data parallelism of the Aho-Corasick algorithm

Snort adopts the Aho-Corasick algorithm to match thousands of attack patterns in parallel. The third level is the data parallelism of the Parallel Failureless Aho-Corasick (PFAC) [7][8] algorithm on GPUs. PFAC is a multi-threaded algorithm for enhancing the Aho-Corasick algorithm. In Snort, attack patterns are grouped by their protocols and the patterns in the same group are compiled into a state machine. An Aho-Corasick state machine contains both valid and failure transitions. For example, Fig. 2 shows the Aho-Corasick state machine used to match the four patterns of "he", "she", "his", and "her", where solid lines represent valid transitions and dashed lines represents failure transitions. Then, multiple string matches are done by traversing the Aho-Corasick state machine. Take the input string "shers" as an example. The state machine starts from the initial state 0. After obtaining "s", the state machine enters state 3. After that, the input character "h" is acquired, and the state machine moves from state 3 to state 4. Next, the input character "e" is obtained and the state machine moves to the final state 5 which represents the pattern "she" is matched. Then, the input character "r" is obtained. Because there is no valid transition for "r", the state machine moves to state 2 by the failure transition. Meanwhile, state 2 represents the pattern "he" is matched. After reaching state 2, the state machine takes the input character "r" again and moves to

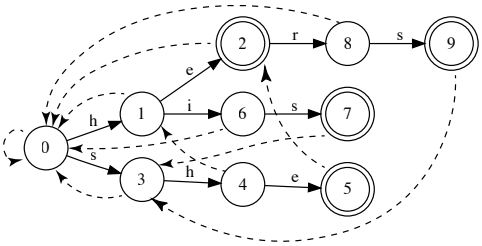

Fig. 2: The Aho-Corasick state machine used to match "he", "she", "his", and "her"

state 8. Finally, after getting "s", the state machine reaches the final state 9 which represents the pattern "hers" is matched.

The Aho-Corasick algorithm uses one thread to scan from the beginning of the input string to the end, by traversing the Aho-Corasick state machines. To accelerate the Aho-Corasick algorithm, we have proposed a fine-grained data parallel algorithm called Parallel Failure-less Aho-Corasick (PFAC) algorithm [7][8]. As shown in Fig. 3, the PFAC algorithm assigns a separate thread for each character position, with each thread responsible for traversing a specific PFAC state machine. The PFAC requires n threads for an input string of length n. Each thread only needs to be responsible for matching any of the signature rules that match the beginning of the character at its location. In other words, each thread only needs to traverse valid transitions. When there is no valid transition, the thread terminates immediately without taking failure transitions. Therefore, the traditional Aho-Corasick state machine can be simplified by removing all failure transitions and the loop-back transition in the initial state. Fig. 4 shows the PFAC state machine, which compiles four patterns, "he", "her", "his" and "her". Compared to Fig. 2, all failure transitions as well as the loop-back transition in the initial state are all removed.

Fig. 5 shows an example of using PFAC to find "she", "he" and "her". Consider an input string containing the substring "shers". Thread n is located at the character "s" and thread $n+1$ is located at the character "h". After obtaining "s", "h", and "e", thread n will reach state 5 which represents the pattern "she" is found. On the other hand, thread $n+1$ will reach state 2 after taking "h" and "e" and will reach state 9 after taking "r" and "s". States 2 and 9 indicate that "he" and "hers" are found, respectively. Different from the traditional Aho-Corasick algorithm, the PFAC adopts multiple threads to search patterns in parallel.

PFAC is a fine-grained parallel algorithm that is suitable for implementation on GPUs. Although PFAC initially requires thousands of threads, most threads terminate very early. The time complexity of PFAC is O(1) to O(m) where m is the length of the longest pattern. Therefore, the overall performance is much better than the traditional Aho-Corasick algorithm implemented on CPUs. We have released the source code of PFAC on Github (https://github.com/pfac-lib/PFAC).

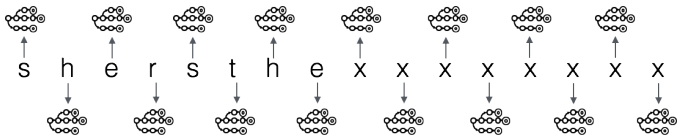

Fig. 3: Parallel Failure-less Aho-Corasick (PFAC) Algorithm

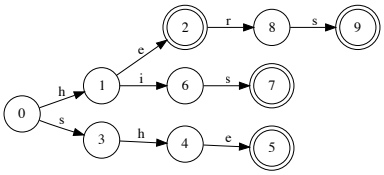

Fig. 4: PFAC state machine

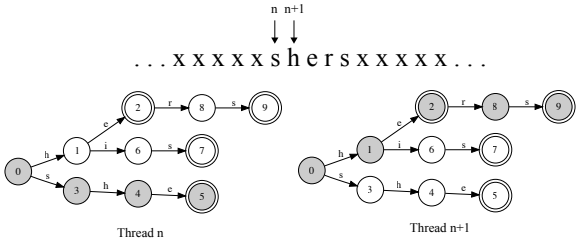

Fig. 5: Example of PFAC

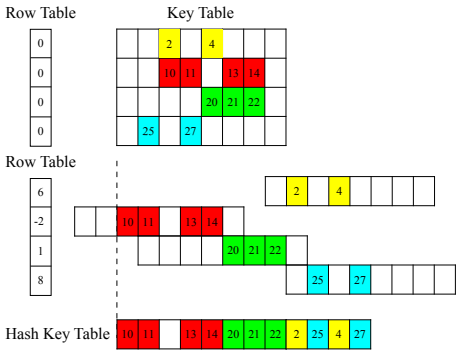

Fig. 6: An example of perfect hashing construction

D. Construction of State Machine

In terms of building the AC state machine, Snort uses linked lists to store the AC state table. There are two disadvantages of the linked lists, one is the inefficiency of traversing the state machine and the other is the memory requirement for storing pointers. In our approach, we use perfect hashing [9] to store the PFAC state machine. Fig. 6 shows an example of using perfect hashing to compress a 2D table. The perfect hashing algorithm first places the 11 valid keys in the 4x8 table in size order. Then, the perfect hashing algorithm moves the four rows to the appropriate places, with no two keys in the same column. The offset of rows are recorded in the row table. Finally, the 2D table can be compressed into a one-dimensional vector stored in the hash key table.

E. Post-processing

As discussed in Section 2, Snort processes individual packets, performs string matching through the Aho-Corasick algorithm [1] in the Detect module, and outputs the result as to where the packet has been successfully matched and sends a warning message to the user. Since it is not effective to process a single packet on GPUs, we propose putting packets into buffers and batch processing buffered packets. When the buffer is full or reaches its maximum elapsed time, the batch of packets is moved to GPUs for string matching. Then, the GPU will return match results, reporting the locations of matched patterns. The problem is that given a match pattern location, how to quickly know which packet contains the pattern. In this paper, we propose to create a balanced binary search tree to record the packet's ID and its location in the buffer. The time complexity of searching a balanced binary tree is O(log$n$), where n is the number of nodes.

IV. Experimental Results

The experimental environment is equipped with a CPU host and 4 GPU devices. The host uses the Intel Xeon E5-2683 CPU, which includes 16 cores operating at 2.1GHz and two hardware threads per core. The device uses NVIDIA GeForce GTX Titan X GPU, which includes 3,840 CUDA cores and 12GB DDR5X DRAM. The operating system is Xubuntu 16.04 LTS. CUDA [11] and OpenMP [12] are adopted to implement the proposed hierarchical parallel approach. The compiler includes GCC-5.4.0 and CUDA-8.0s NVCC.

First, we compare the proposed hierarchical PFAC with the traditional AC algorithm used in Snort. We use the packet generator to generate packets of different throughput as input packets. The input size is 208MB, with a minimum of 2.4 Gbps and a maximum of 20 Gbps, respectively, as shown in Table 1. We extract the Blacklist rules from Snort 2.9.x as the pattern rule file. The blacklist rules consist of 1,000 rules, 29,949 characters, a total of 30K bytes. The PFAC with 4 GPUs achieves an average throughput of 70 Gbps, which is more than 40 times faster than the traditional AC that achieves average throughput of only 1.63 Gbps. Table 1 shows that the PFAC achieves significant performance improvements over the traditional AC that does not meet packet throughput requirements. Table 2 shows the comparison with the related approaches. Our proposed method shows better performance than the other methods.

In terms of memory requirements, for the blacklist rules, Snort needs a link list of 21.8 MB, whereas we only need a perfect hash table of 0.1725 MB. The proposed approach achieves up to 99.2% of memory reduction on Snort.

V. Conclusion

We have proposed a hierarchical parallel architecture that consists of three levels of parallelism to accelerate an open-source NIDS, Snort. The hierarchical parallel architecture has achieved significant improvements in performance and memory reduction.

TABLE 1: A comparison of AC and PFAC in different packets throughput with four GPUs

Version	Packets number	Packets throughput	AC throughput	PFAC throughput
V1	220978	2.40 Gbps	1.50 Gbps	71.61 Gbps
V2	285044	3.10 Gbps	1.55 Gbps	68.93 Gbps
V3	217913	4.30 Gbps	1.51 Gbps	69.59 Gbps
V4	240717	5.80 Gbps	1.70 Gbps	68.77 Gbps
V5	222444	6.35 Gbps	1.52 Gbps	69.36 Gbps
V6	222431	6.40 Gbps	1.69 Gbps	70.55 Gbps
V7	218164	8.40 Gbps	1.69 Gbps	70.20 Gbps
V8	217510	9.80 Gbps	1.71 Gbps	71.50 Gbps
V9	217135	16.0 Gbps	1.76 Gbps	70.14 Gbps
V10	214913	20.0 Gbps	1.71 Gbps	72.25 Gbps

TABLE 2: Comparison with other methods

Paper	Platform	Algorithm	System throughput
Para-Snort [5]	CPU	AC	0.843 Gbps
Gnort [6]	GPU	AC	2.300 Gbps
Z.K. Baker [10]	FPGA	KMP	6.400 Gbps
A. Mitra [4]	FPGA	AC	12.90 Gbps
Our method	GPU	PFAC	70.29 Gbps

References

[1] Alfred V. Aho , Margaret J. Corasick, "Efficient string matching: an aid to bibliographic search," Communications of the ACM, v.18 n.6, p.333-340, June 1975

[2] M.Roesch. Snort-lightweight intrusion detection for networks. In the 13th USENIX Conference on System Administration, 1999.

[3] OSEC - http://osec.neohapsis.com

[4] Abhishek Mitra , Walid Najjar , Laxmi Bhuyan, "Compiling PCRE to FPGA for accelerating SNORT IDS," Proceedings of the 3rd ACM/IEEE Symposium on Architecture for networking and communications systems, December 03-04, 2007, Orlando, Florida, USA

[5] X. Chen, Y. Wu, L. Xu, Y. Xue, and J. Li., "Para-Snort: A Multi-thread Snort on Multi-core IA Platform.," In Proceedings of Parallel and Distributed Computing and Systems (PDCS), 2009.

[6] Vasiliadis G, Antonatos S, Polychronakis M, Markatos EP, Ioannidis S, "Gnort: high performance network intrusion detection using graphics processors," In: 11th international symposium on recent advances in intrusion detection, Boston, MA, USA, 2008, pp. 116-134.

[7] C.-H. Lin, J.C. Li, C.H. Liu, S.C. Chang, "Perfect hashing based parallel algorithms for multiple string matching on graphic processing units.," IEEE Trans. Parallel Distrib. Syst. 99, 1 (2017)

[8] C.-H. Lin, C.-H. Liu, L.S. Chien, S.C. Chang, "Accelerating pattern matching using a novel parallel algorithm on gpus.," IEEE Trans. Comput. 62(10), 19061916(2013)

[9] C.-H. Lin, C.-H. Liu, S.-C. Chang, and W.-K. Hon, "Memory- efficient pattern matching architectures using perfect hashing on graphic processing units," in Proc. 31st Annu. IEEE Int. Conf. Com- put. Commun., 2012, pp. 1978-1986.

[10] Z. K. Baker and V. K. Prasanna, "A methodology for synthesis of efficient intrusion detection systems on FPGAs.," Proc. FCCM, 2004.

[11] NVIDIA: CUDA Zone (2016). https://developer.nvidia.com/cuda-zone

[12] The OpenMP API specification for parallel programming (2016). http://www.openmp.org/